A Model for the Analysis of Fault-Tolerant Signal Processing Architectures

V. S. S. Nair and J. A. Abraham

Computer Systems Group
Coordinated Science Laboratory
University of Illinois
Urbana, IL 61801

ABSTRACT

This paper develops a new model, using matrices, for the analysis of fault-tolerant multiprocessor systems. The relationship between processors computing useful data, the output data, and the check processors is defined in terms of matrix entries. Unlike the matrix based models proposed previously for the analysis of digital systems, this model uses only numerical computations rather than logical operations for the analysis of a system. We present algorithms to evaluate the fault detection and location capability of the system. These algorithms are much less complex than the existing ones. We also use the new model to analyze some fault-tolerant architectures proposed for signal processing applications.

1. INTRODUCTION

High performance signal processing architectures have the potential for low reliability because of their complex hardware structures, high rate and volume of data processed, and because of long, continuous processing times. An important consideration in the design of such high performance multiple processor systems should be to ensure the correctness of the results computed in the presence of transient and intermittent failures. Concurrent error detection and correction have been applied to achieve reliable computing. To support such on-line checking, the data output from the system can be encoded at a high level and the correctness of the coded output verified by check processors.

Algorithm Based Fault Tolerance (ABFT), suggested by Huang and Abraham [1], deals with a system level method of achieving fault tolerance at a low cost. Original algorithms are modified to operate on encoded data so that it produces encoded output, from which the useful information can be recovered very easily. The task distribution among the processing elements may be done in such a way that any malfunction in a processing element will affect only a small portion of the data, which can be detected and corrected using the properties of the code. ABFT techniques have been proposed for many signal processing computations such as matrix operations [2], Fast Fourier Transforms [3], QR Factorization, and Singular value decomposition [2]. Fault diagnosis schemes, which are an integral part of fault-tolerant computation in all the above cases, are very specific to the algorithms and the architecture used. Due to the increasing popularity of ABFT applications it would be desirable to have a general scheme for fault analysis in these applications. For such a scheme, one needs a general model for representing systems using ABFT. The first attempt towards this was done by Banerjee and Abraham [4] who proposed a graph-theoretic model for representing ABFT systems. This model was based on existing models for multiprocessor systems in which processors mutually checked each other [5,6,7].

In the graph-theoretic model the system is represented by a tripartite graph having three groups of nodes: nodes of type F corresponding to the possible faulty processors, nodes of type E corresponding to the output data elements on which the errors may occur, and nodes of type C corresponding to the checks. There is an edge from an F node i to an E node j if data element d_j is affected by processor P_i. There is an edge from node j of type E to node k of type C if the data element d_j is checked by check c_k. For the analysis of faults in the system, a generalized error table (GET array) is constructed from the graph model [4]. The GET array contains all possible error combinations of the faults under consideration. The detectability or locatability of a fault is determined by observing whether all the error patterns produced

This research was supported in part by the National Aeronautics and Space Administration (NASA) under Contract NAG 1-613, and in part by SDIO/IST and managed by the Office of Naval Research (ONR) under contract N00014-86-K-0519.

by that fault are detected or located by the checks provided.

Even though this model can be used for the accurate analysis of systems using ABFT, it has some drawbacks. It is very tedious to analyze a system using this model since it requires the exhaustive enumeration of all fault and error combinations. This leads to enormous memory and time requirements for the analysis of complex systems with a large number of processors, with each processor producing a large volume of data. The second problem is with the parameter definitions of the model. Fundamental system parameters such as closure index, masking index, and exposure index are defined very similar to the parameters with the same name in the Russel and Kime model for fault diagnosis [6,7]. Unfortunately some of those parameters (for example exposure index) do not have any physical significance in a multiple processor system such as the one described in [4]. Finally, the model cannot be easily used for the design of a system for a given error detectability and correctability.

In this paper we develop a new model, using matrices, for the analysis and design of fault-tolerant multiple processor systems. In the conventional analysis of such systems, one assumes the existence of complete tests which always fail when there is a faulty processor [6]. A processor can test another, and if the tested unit is faulty and the tester is fault-free, then the test is guaranteed to fail. However, in systems using ABFT, a particular fault pattern can produce a number of different error patterns. The checking operations detect the errors directly and the faults indirectly. Therefore, fault analysis in such systems is much more complex than conventional fault analysis. The new model is specifically developed for analyzing faults in systems using ABFT; however, application of the model for fault analysis of conventional redundancy techniques (such as duplication or triplication) is trivial.

We define three matrices, the PD (Processor-Data) matrix, the DC (Data-Check) matrix and the PC (Processor-Check) matrix, which describe the system as a whole. The model is described in detail in section 2. In section 3 we describe algorithms for the analysis of systems for fault detectability and locatability. Unlike the existing algorithms, the new algorithms do not need exhaustive enumeration of faults and errors. This is achieved through an error collapsing technique using the proposed model. The model can also be used for the design of fault-tolerant systems. Applications of the model to analyzing some fault-tolerant signal processing architectures are given in section 4.

2. THE NEW MODEL

2.1. System description

In the multiprocessor system under consideration, the computation is done on m processors $P_1, P_2,..., P_m$. These processors generate n data elements d_1 through d_n as results. If processor P_i produces or affects data d_j, this is represented as $d_j \in DATA(P_i)$. The data elements are encoded at a high level using some encoding techniques suitable for the particular algorithm [8]. Once the actual data computation is complete, the output is checked for the correctness based on the fault-tolerant encoding scheme. The system is assumed to have l checks, C_1 through C_l, done on the output. The new model for the system is based on the following assumptions regarding the presence of a fault.

A fault is defined to be a condition that may cause a malfunction of a particular processing unit. The fault may be transient or permanent. An error is any discrepancy between the expected results of an operation and the actual operation under the faulty conditions. Therefore, an error implies the presence of a fault in the computing system. At the same time, a fault in a processing unit might not produce an error in the results for certain combinations of inputs. Since the effect of such a fault is not observable in the form of an error, we consider that as a fault-free situation. In other words, in our analysis, a processor is said to be faulty if and only if there exists at least one error in the data computed by that processor.

As mentioned above, checking for the correctness of the output is made possible through high level data encoding. The algorithm for the actual data computation is modified to operate on the encoded data. The number of errors which can be detected and corrected using the particular check depends on the properties of the code.

DEFINITION 1: The error detectability of a check is h if and only if the check can detect the presence of an error, provided the number of erroneous data elements in the data set on which the check is done, do not exceed h.

If the error detectability and locatability of the checks are specified, the fault detectability and locatability of the system depend on the way the checks are distributed among the output.

2.2. The Model Matrices

In the new model for multiple processor systems, the relation between processors, data, and checks are represented by three fundamental matrices, the PD (Processor-Data) matrix, the DC (Data-Check) matrix, and the PC (Processor-Check) matrix.

DEFINITION 2: The PD matrix is an $m \times n$ matrix such that

$$PD_{ij} = \begin{cases} 1 & \text{if } d_j \in DATA(P_i) \\ 0 & otherwise \end{cases}$$

DEFINITION 3: The DC matrix is an $n \times l$ matrix such that

$$DC_{ij} = \begin{cases} 1 & \text{if } C_j \in CHECK(d_i) \\ 0 & otherwise \end{cases}$$

DEFINITION 4: The PC matrix is an $m \times l$ matrix which is the product of PD and DC matrices.

EXAMPLE: Consider a system described as
$DATA(P_1) = \{d_1, d_2, d_3\}$
$DATA(P_2) = \{d_2, d_4\}$
$DATA(P_3) = \{d_5\}$
$DATA(P_4) = \{d_6\}$
$CHECK(d_1) = \{C_1\}$
$CHECK(d_2) = \{C_2\}$
$CHECK(d_3) = \{C_1\}$
$CHECK(d_4) = \{C_2, C_3\}$
$CHECK(d_5) = \{C_1\}$
$CHECK(d_6) = \{C_2, C_3\}$.
Then the fundamental matrices are

$$PD = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad DC = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

$$PC = PD \times DC = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

2.3. Physical significance of fundamental matrices

The physical significances of the PD and the DC matrices are clear from their definitions. In the PC matrix, PC_{ij} represents the number of data elements of P_i checked by check C_j. It can be noticed that entries in PD and DC matrices are either a 0 or a 1, whereas PC matrix can have elements as large as n.

The importance of these matrices in the analysis of faults in the system will be revealed in the following discussion. Without loss of generality we can use the same matrices for representing faults and errors in the system. The only difference is that in the PD and PC matrices, the row corresponding to P_i stands for a fault in processor P_i. Those elements of row P_i of the PD matrix will be 1 if the corresponding data elements are erroneous due to a fault in processor P_i.

$$PD_{ij} = \begin{cases} 1 & \text{if } d_j \textit{ is erroneous when } P_i \textit{ is faulty} \\ 0 & otherwise \end{cases}$$

The PD matrix will be different for different error combinations at the output. Correspondingly we will also have different PC matrices. The DC matrix will be invariant, determined only by the system designer. It is easy to observe that during the analysis of faults in the system, each row in the fundamental PD matrix, defined earlier, represents a faulty processor whose output data elements are all wrong.

3. FAULT DIAGNOSIS OF THE SYSTEM

In this section we describe two algorithms for analyzing the system for fault detectability and locatability using our new model.

DEFINITION 5: A fault-tolerant system has *t–fault detectability* if and only if some check C_i will definitely fail, provided the number of faults present in the computing system does not exceed t.

3.1. Analysis for fault detectability

For the analysis we define matrices ^{r}PD and ^{r}PC which are derived from the PD and the PC matrices respectively.

DEFINITION 6: ^{r}PD is defined as the matrix whose rows are formed by adding r different rows of matrix PD, for all possible different combinations of r rows, and then setting all nonzero elements to 1.
Note that a nonzero element greater than 1 results from the addition of rows of the PD matrix if the processors corresponding to those rows have common data elements. These nonzero elements are set to 1 in order to avoid duplication of the same data element.

DEFINITION 7: ^{r}PC is defined as the matrix whose rows are formed by adding r different rows of matrix PC, for all possible different combinations of r rows.
^{r}PC is an $\binom{m}{r} \times l$ matrix. In the fault analysis, each row of ^{r}PD and ^{r}PC will represent the situation in which r faults are present simultaneously. As a special case it may be observed that ^{1}PC = PC.

DEFINITION 8: The row R of ^{r}PC is said to be completely detectable if and only if the fault represented by R is detectable for all possible error patterns produced by that fault.
That is, there should be at least one element in the row R which is less than or equal to the error detectability (h) of the check used, for all possible errors. If we enumerate all the possible error combinations, the algorithm to check the complete detectability of a row will be as complex as the previous ones. Instead, we use an error collapsing technique so that the algorithm converges much faster and needs less storage.

3.1.1. Algorithm to check whether R is completely detectable

ALGORITHM 1.

(1) If there is no element in row R which is less than or equal to h and greater than 0, R is not completely detectable, Stop. Otherwise go to step 2.

(2) Find all j such that $0 < R_j \leq h$

(3) If $DC_{ij} = 1$ set ${}^{r}PD_{Ri} = 0$. Also set the corresponding elements in the i^{th} column of PD matrix to zero. Here $i = 1, 2, \ldots l$. Do the same for all j obtained from step 2.

(4) If at least one row of new PD matrix has all zeros, R is competely detectable, Stop. Otherwise go to step 5.

(5) Find the new ^{r}PC matrix by multiplying the new ^{r}PD matrix obtained from step 3 with the DC matrix and go to step 1.

In the following discussion we will describe how the algorithm works. In the first step, if the entries of R are all zeros, a fault corresponding to R is not observed as detectable errors. On the other hand, if some of the entries are greater than h, it means that the errors caused by the fault invalidate the corresponding checks. In either case, the fault is not detectable. As mentioned before, in the case of analysis of faults in systems, the fundamental matrices PD and PC represent the situation in which all the output elements of a faulty processor are erroneous. In the algorithm we start with a row R which is a combination of some rows of the PC matrix. Therefore, R represents a fault such that the output data

elements of processors associated with R all erroneous. If at least one element of R is less than or equal to h and greater than zero (we call such an entry a "valid entry") we can conclude that the fault is detectable by some checks provided all the data elements from the faulty processors are erroneous. This does not imply that the fault will be detected for all possible error combinations.

For example, consider a system in which

$DATA(P_1) = \{d_1, d_2\}$
$DATA(P_2) = \{d_3\}$
$CHECK(d_1) = CHECK(d_3) = \{C_1\}$
$CHECK(d_2) = \{C_2\}$

Then we have the fundamental matrices

$$PD = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad DC = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \quad PC = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Obviously the system is single fault detectable if h=1. In order to check whether the system is 2–*fault detectable* we compute ^{2}PC which is equal to [2, 1]. Since there is a 1 in this row, the fault is detectable when all the data elements (d_1, d_2, and d_3) are erroneous. Now consider the situation in which the faulty processor P_1 produces an error in d_1 alone. A fault in P_2 will definitely cause an error in d_3. Thus errors in d_1 and d_3 will invalidate the check C_1; at the same time, check C_2 will pass since the data d_2 is not erroneous. As a result if P_1 and P_2 are faulty, the fault may not be detected, and hence the system is not 2–*fault detectable.*

This discrepancy is taken care of in step 3 of Algorithm 1. The objective is to check whether the fault is detectable for all possible output error combinations. For that, we use a technique called error collapsing. All the elements of ^{r}PD which contributed to the valid entries of row R are set to zero. By doing this, effectively we are removing those errors which are detectable at the output. We may remove all those errors simultaneously, because if at least one of them were not removed, that would be detectable at the output and hence the fault is detectable.

When we collapse errors in ^{r}PD, we have to do the same for the original PD matrix. After that, one has to check whether every row of the PD matrix has at least one nonzero entry in order to ensure that every faulty processor produces at least one erroneous output data element. The new ^{r}PC is calculated by multiplying the ^{r}PD matrix obtained after error collapsing with the DC matrix. This new matrix will be different from the old ^{r}PC in two ways. The new matrix will have zeros in the corresponding positions where the old ^{r}PC had valid entries. Also some of the invalid entries in the old matrix might have become valid entries in the new matrix. This is because removal of errors may make some of the invalid checks valid.

This iteration is done as given in Algorithm 1 to check the complete detectability of row R.

DEFINITION 9: The matrix ^{r}PC is said to be completely detectable if and only if all rows of ^{r}PC are completely detectable.

THEOREM 1: A fault-tolerant system is t–*fault detectable* if and only if the matrices ^{i}PC, for $i = 1, 2, \ldots t$, are completely detectable.

PROOF: Proof of the theorem follows from the definitions 5, 6, 7, 8, and 9. □

The complexity of the algorithm used is $O\left(\sum_{i=1}^{i=t+1} \binom{m}{i}\right)$. This is much smaller than the complexity of the algorithm using graph theoretic models [4], which is equal to $O\left(\sum_{i=1}^{i=t+1} \binom{2^n}{i}\right)$.

3.2. Analysis of the system for fault locatability

Analyzing the system for its fault locatability is a much harder problem when compared to the problem of finding the fault detectability. This is because, in the case of locatability, we have to check not only whether some faults are detected, but also whether that fault is distinguishable from other faults.

DEFINITION 10: A system is said to have t–*fault locatability* if and only if the application of the check set identifies precisely which faults are present, provided the number of faults does not exceed t.

LEMMA 1: A necessary condition for *t–fault locatability* for $t \geq 1$ of a system is that that $\Sigma_i PD_{ij} \leq 1$.

PROOF: We may prove the lemma by contradiction. $\Sigma_i PD_{ij} \leq 1$ implies that there can be at most one 1 in every column of the PD matrix which means $DATA(P_i) \cap DATA(P_j) = \phi$ for all $i \neq j$. If possible, let there be more than one 1 in a column, which implies that data sets produced by certain processors are not disjoint. In that case if an error is observed in the common data element or elements, it will not be possible to conclude which faulty processor produced that error. Therefore, the system is not single fault locatable. So, the assumption that there is more than one 1 in a column is wrong, and hence the proof. □

In most of the existing multiprocessor systems, processors have nondisjoint output data sets so that the assumption $DATA(P_i) \cap DATA(P_j) = \phi$ is not valid. In those systems, locating a faulty processor will not be possible according to Lemma 1. However, processors whose data sets have nonempty intersections with other processors can be collapsed into *processor classes* [4] so that the *processor classes* will have disjoint data sets. One may be able to locate a faulty *processor class* (a *processor class* is said to be faulty if at least one of the processors in the class is faulty) during the fault diagnosis of the system. The PD matrix for the *processor classes* are found by adding together the rows of the PD matrix corresponding to the processors in the *processor class* and by setting all nonzero elements to 1.

In the example given in Section 2.2 for the model matrices of a system, $DATA(P_1) \cap DATA(P_2) = \{d_2\}$. Here, P_1 and P_2 will form a *processor class*. P_3 and P_4 form two different *processor classes*. Now the corresponding PD and PC matrices are

$$PD = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad PC = \begin{bmatrix} 2 & 2 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

For convenience, in the forthcoming discussion, we use the term *locatability* to mean *class locatability*.

DEFINITION 11: Row R_1 and R_2 of matrix ${}^1PC$ are said to have $1-0$ *disagreement* if there is at least one valid element in row R_1 such that R_2 has a zero in the corresponding position.

DEFINITION 12: Row R_1 and R_2 of matrix ${}^1PC$ are said to have $0-1$ *disagreement* if there is at least one 0 in either row R_1 or R_2 such that the other row has a valid element in the corresponding position.

DEFINITION 13: If all pairs of rows of ${}^1PC$ have $0-1$ *disagreement*, then ${}^1PC$ is said to have a $0-1$ *disagreement*.

DEFINITION 14: PC has $1-0$ *disagreement* with ${}^1PC$ if and only if every row R of PC has $1-0$ *disagreement* with all rows of ${}^1PC$ which do not contain R.

It may be noted that a $1-0$ *disagreement* implies a $0-1$ *disagreement*, whereas a $0-1$ *disagreement* does not imply $1-0$ *disagreement*. That is $1-0$ *disagreement* is a stronger condition than $0-1$ *disagreement*.

3.2.1. Physical significance of disagreement

When two rows of ${}^rPC$ have $0-1$ *disagreement*, that means the faults corresponding to those rows are distinguishable provided the output from the processors involved in those faults are all erroneous. A $1-0$ *disagreement* between PC and ${}^rPC$ will imply that every individual fault is exposed (or not masked) in all fault patterns of cardinality $r+1$. This is because every row R in PC has a $1-0$ *disagreement* with all rows of ${}^rPC$ which do not contain R. In both of the above cases we need all data outputs from a faulty processor to be erroneous, which may not be the case all the time. So, we define a stronger relation between rows, namely *complete disagreement*.

DEFINITION 15: A disagreement ($0-1$, or $1-0$) between two rows is called a *complete disagreement*, if the disagreement exists for all possible error combinations caused by the faults associated with those rows.

In order to check for the *complete disagreement* between two rows we use an algorithm very similar to the one used for finding the complete detectability. The procedure is outlined in Algorithm 2. Whenever there is a disagreement, the valid entry or entries which caused the disagreement are set to zero by error collapsing as described in Algorithm 1. The algorithm always converges because removal of an error will never convert a 0 to a 1 or a higher value, whereas it may or may not decrease the values of non-zero entries.

ALGORITHM 2.

Input to the algorithm are rows R_1 and R_2 whose complete disagreement are to be checked.

(1) Check whether R_1 and R_2 have a disagreement. If not, output *NO*, Stop. Otherwise go to (2).

(2) Collapse errors and check whether any row of PD matrix has all zeros. If so, output *YES*, Stop. Else go to (1).

(In the latter part of the paper, whenever we talk about a *disagreement* we mean a *complete disagreement*.) Now we prove some necessary and sufficient conditions for t-fault locatability of a system, and develop an algorithm for the analysis.

DEFINITION 16: A fault is said to be exposed under a multiple fault situation, if the fault is not masked by other faults present in the system.

LEMMA 2: A sufficient condition for t-fault locatability of a system is that in any fault pattern of cardinality k, min(k, 2t+1-k) faults are exposed for $k = 1, 2, \ldots \min(2t,n)$

PROOF: Proof of the lemma is same as that of Theorem 2 of [7]. □

In the lemma, we observe that whenever the cardinality of the fault pattern is $\leq t$, all individual faults should be exposed in order for the fault pattern to be locatable. When the cardinality is $2t - r$ for $0 < r < t$, a minimum of $r + 1$ faults should be exposed. In order to check whether the system is t-fault locatable, we have to consider all fault patterns of cardinality $\leq 2t$. We prove a simpler sufficient condition for *t–fault locatability* so that we need to consider only fault patterns of cardinality at most t.

THEOREM 2: A necessary and sufficient condition for *t–fault locatability* is that all individual faults are exposed in every fault pattern of cardinality $\leq t$, and all fault patterns of cardinality t are distinguishable from each other.

PROOF: We prove this theorem through a simple construction. We use rectangles to represent faults. The length of the rectangle corresponds to the cardinality of the fault pattern. When two fault patterns have common individual faults, the rectangles overlap in their positions. Consider two faults F_1 and F_2 whose cardinalities are $\leq t$.

Case 1.

Let the cardinality of $F_1 = |F_1| = t$ and $F_2 \subset F_1$. The representation using rectangles is shown in Figure 1. Since all individual faults in F_1 are exposed, there is a $0 - 1$ *disagreement* between regions A and B. But $F_2 = B$ which implies F_1 has a $0 - 1$ *disagreement* with F_2. That is, F_1 and F_2 are distinguishable.

Case 2.

Let $|F_1| = t$ and $|F_2| = r < t$. Also $F_1 \cap F_2 = \emptyset$. We augment F_2 with some faults contained in pattern F_1 such that the augmented F_2 (F'_2) has cardinality t. Now F_1 and F'_2 are distinguishable by the assumption of the theorem. That is, region A has a $0 - 1$ *disagreement* with region C. (Note that since region B is common to both F_1 and F'_2, it will not contribute to the distinguishability of the faults.) But $C = F_2$ and hence F_1 and F_2 are distinguishable.

Case 3.

Let $|F_1| < t$, $|F_2| < t$ and $F_1 \cap F_2 \neq \emptyset$, also $|F_1 \cup F_2| > t$. (This is the most general case.) In the figure we construct a fault F'_1 by augmenting F_1 with some faults (C') from region C such that $|F'_1| = t$. Similarly F'_2 is constructed by adding a portion of A (A') to F_2. Now F'_1 and F'_2 are distinguishable, which means regions $A - A'$ (part of region A which contains the faults which are not contained in A') and $C - C'$ have a $0 - 1$ *disagreement*. But $A - A' \subset F_1$, and $C - C' \subset F_2$. Therefore, F_1 and F_2 have a $0 - 1$ *disagreement* and are hence distinguishable.

Thus any two fault patterns of cardinality $\leq t$ are distinguishable and hence the sufficiency condition is proved. Proof for the necessary condition follows from the definition of *t–fault locatability*. □

The above results may be translated into the domain where we use the new model for the analysis of fault-tolerant systems.

THEOREM 3: A given fault-tolerant system is *t–fault locatable* if and only if matrices PC and ^{i}PC, for $i = 1, 2, \ldots (t-1)$, have $1 - 0$ *disagreement*, and ^{t}PC has $0 - 1$ *disagreement* with itself.

PROOF: The condition that PC has $1 - 0$ *disagreement* with ^{i}PC implies that all individual faults are exposed in fault patterns of cardinality $\leq t$. Since ^{t}PC has $0 - 1$ *disagreement* with itself, all fault patterns of cardinality t are distinguishable. Hence the system is *t–fault locatable* by Theorem 2. □

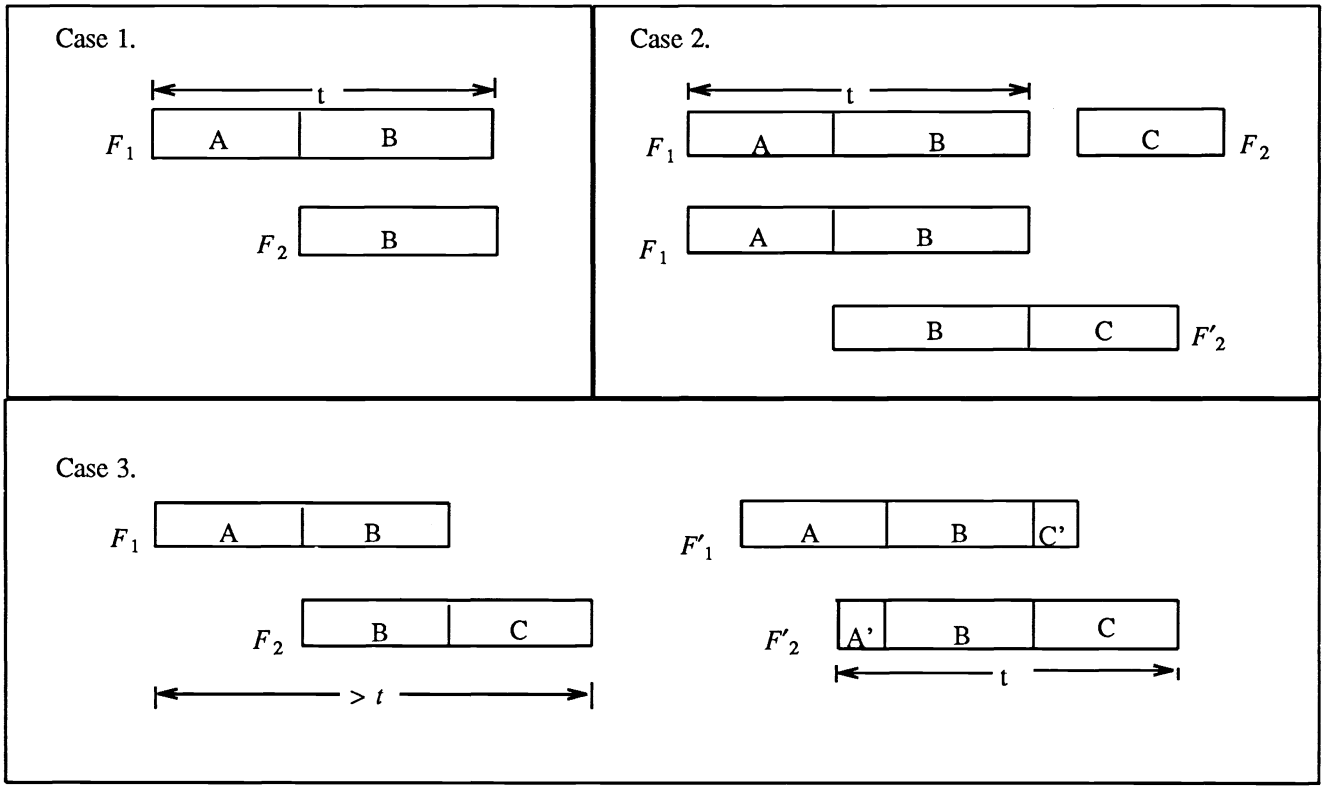


Figure 1. Fault patterns of cardinality $\leq t$

The new procedure to find the fault locatability of the system is less complex than the previous algorithms, due to two reasons. First, we use the error collapsing technique which avoids a possible combinatorial explosion. Second, by stating a new sufficient condition we have avoided enumerating $\sum_{i=1}^{i=2t} \binom{m}{i}$ number of fault patterns and need to consider only $\sum_{i=1}^{i=t} \binom{m}{i}$ patterns instead. (Here we assume that $2t \ll m$.)

4. MODEL APPLICATIONS

In this section we apply the new model for the analysis of some signal processing architectures which make use of ABFT.

EXAMPLE 1: Consider matrix multiplication using checksum on a mesh connected processor array. The fault tolerance scheme had been proposed by Huang and Abraham [1]. We will briefly describe the system below.

Multiplication of two 3×3 matrices X and Y is done on a mesh connected processor array as shown in Figure 2(a). We assume that input data elements are broadcast on the buses; the processors input the data from the buses. Under a processor failure, we assume that only the corresponding data element of the output matrix Z becomes erroneous. After the computation, the result Z resides in the local memories of the processors. Now the checking operations are (six of them, three for rows and three for columns) performed.

Thus we have

$DATA\ (P_i) = d_i$ for $i = 1, 2, \ldots 9$.

$CHECK(d_1, d_2, d_3) = C_1$ $\quad$ $CHECK(d_4, d_5, d_6) = C_2$

$CHECK(d_7, d_8, d_9) = C_3$
$CHECK(d_1, d_4, d_7) = C_4$
$CHECK(d_2, d_5, d_8) = C_5$
$CHECK(d_3, d_6, d_9) = C_6$

Now the fundamental matrices of the system will be

$PD = I_9$, where I_9 is the identity matrix of order 9.

$$DC = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$PC = PD \times DC = DC$, since $PD = I_9$.

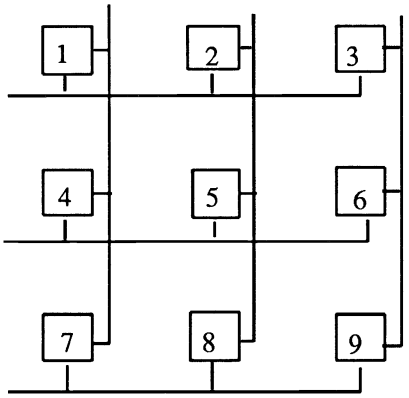
During the analysis of the system for *t–fault detectability* we see that ^{r}PC is completely detectable for $r = 1, 2, 3$. In ^{4}PC the row R corresponding to the sum of rows P_1, P_2, P_4 and P_5 of PC is [2 2 2 2 0]. Since error detectability of checksum encoding is 1 (ie., h=1) none of the elements in R is valid. Therefore R and hence ^{4}PC is not completely detectable. Then by Theorem 1 the system is 3–*fault detectable*.

Next we will consider the fault locating capability of the algorithm. Since the matrix PC has a 0 – 1 *disagreement* with itself, the algorithm is 1–*fault locatable*. In the analysis procedure we observe that PC does not have 1 – 0 *disagreement* with ^{2}PC. For example row P_1 of PC is [1 0 0 1 0 0] and this does not have a 1 – 0 *disagreement* with the row P_2P_4 (ie., the sum of P_2 and P_4) of ^{2}PC which is equal to [1 1 0 1 1 0]. Hence the system is at most 2-fault locatable. As a next step we check whether all faults of cardinality 2 are distinguishable. For that, ^{2}PC should have 0 – 1 *disagreement* with itself. One can observe that rows P_1P_5 = [1 1 0 1 1 0] and P_2P_4 = [1 1 0 1 1 0] of ^{2}PC do not have 0 – 1 *disagreement*. Hence ^{2}PC does not have 0 – 1 *disagreement* and the system is 1–*fault locatable* by Theorem 3.

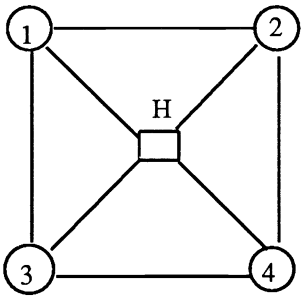
EXAMPLE 2: In this example we analyze an algorithm for fault-tolerant matrix multiplication using checksum encoding done on a hypercube. We consider partitioned matrix multiplication done on a 4 node hypercube as shown in Figure 2 (b). In the figure, the circles represent the hypercube nodes and the square represents the host processor. In the fault-tolerant algorithm suggested in [9], processor 1 checks the correctness of the data computed by processor 2 and sends a "pass" or "fail" signal to the host processor. At the same time processor 2 checks the data computed by processor 1 and sends a signal to the host. Similarly processors 3 and 4 also check each other and notify the host of the result.

Here, even though every processor P_i for $i = 1, 2, 3, 4$, produces data d_i, it is not necessary to include them in the PD matrix. This is because the check by the host processor is done only on the flag signals generated by the node processors. Let e_i be the flag signal generated by P_i. Then, from the description of the algorithm, we have
$CHECK(e_1, e_2) = C_1$
$CHECK(e_3, e_4) = C_2$
Note that both the checks are resident in the host processor. Now the PC matrix is

$$PC = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}$$



(a) Mesh connected array



(b) 4 node hypercube

Figure 2. Processor arrays.

It can be seen that the system is 1–*fault detectable* and 0–*fault locatable*. However, if the mutual checking processor pairs are changed in the fault-tolerant algorithm, that is instead of 1,2 and 3,4, if checking is done by pairs 1,3 and 2,4 and flag signals sent to the host processor, in effect we are adding two more checks given by

$CHECK(e_1, e_3) = C_3$

$CHECK(e_2, e_4) = C_4$

The new PC matrix is

$$PC = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

Carrying out a similar analysis we found that the the algorithm is 3–*fault detectable* and 1–*fault locatable*. This example was specifically chosen in order to illustrate the importance of selecting data elements for the PD matrix so that the analysis will be easier.

EXAMPLE 3: As a final example we consider the Advanced On board Signal Processor (AOSP) architecture [10]. The AOSP is an architectural concept for an advanced signal processing computer that provides a fault-tolerant environment capable of supporting a wide range of signal processing applications. It is a loosely-coupled distributed multiprocessor system in which a large number of identical processors known as Array Computing Elements (ACEs) communicate both data and control information via packetized messages over networks of high-speed buses.

In order to achieve fault tolerance, one may incorporate some kind of system level fault diagnosis in the AOSP architecture. In this example we consider an AOSP with system level fault diagnosis. The encoding scheme to be used depends on the particular signal processing application for which AOSP is used. In the example we do not assume any particular computation or encoding scheme. The only assumption made is that the encoding scheme can detect one error. (ie., h=1)

The physical architecture of the AOSP is depicted in Figure 3. Due to the high density of the interconnection network, we have the luxury of having a fault-tolerant scheme in which any arbitrary processor can check the correctness of the computation done by any other processor. As an example we consider a scheme in which

$CHECK(d_8, d_4, d_3) = C_1$

$CHECK(d_8, d_6, d_1) = C_2$

$CHECK(d_4, d_2, d_9) = C_3$

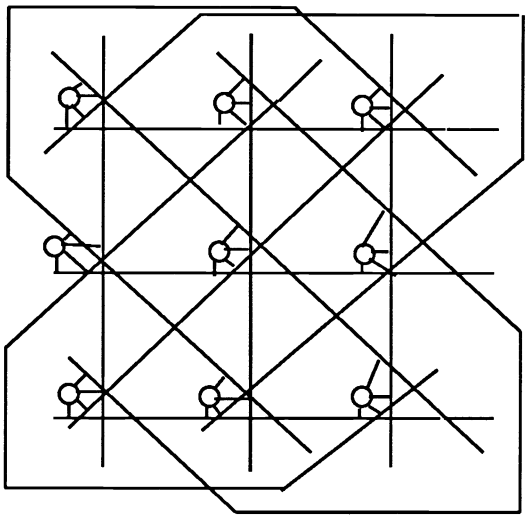


Figure 3. AOSP Architecture.

$CHECK(d_1, d_5, d_9) = C_4$
$CHECK(d_3, d_5, d_7) = C_5$
$CHECK(d_1, d_5, d_9) = C_6$

By the analysis using the fundamental matrices we find that the system is 3–*fault detectable* and single fault locatable.

5. DESIGN APPLICATIONS

The matrix based model suggested in the paper can also be used for the design of fault-tolerant systems. The design objective is to find out how to allocate checks efficiently in order to achieve the required fault detectability and locatability, once the actual computation architecture is specified.

By reversing the procedure of finding the detectability and locatability, one may construct a PC matrix for a given fault diagnostic capability. The PD matrix is fixed by the given computing architecture. From these two matrices we can find a DC matrix, since the entries of this matrix are either a 0 or a 1.

In the AOSP example we were allocating the checks arbitrarily. Instead, if we had started with the PC matrix we could have selected the rows in such a way that ^{2}PC has $0-1$ *disagreement* so that the system would have been 2–*fault locatable*. For example, if PC matrix is selected such that $CHECK(d_8, d_6, d_1, d_2) = C_2$, then ^{2}PC will have a $0-1$ *disagreement* and hence the system will be 2–*fault locatable*.

This is a very simple example of the design application of the model. For complex systems, the selection of the PC matrix will not be easy. In such cases one needs a more systematic algorithm, which we plan to develop in our future work.

6. CONCLUSIONS

We developed a new model, using matrices, for fault-tolerant multiple processor systems which use system level fault diagnosis schemes. For determining the fault detection and location capability of the system, we suggested two algorithms which are less complex than the existing algorithms. These algorithms are easily programmable since they consist of simple matrix operations. Applications of the model for the analysis of some fault-tolerant signal processing architectures are also presented. It is hoped that a systematic algorithm for the design of fault-tolerant systems could be developed using the new model.

REFERENCES

[1] J. A. Abraham and Kuang-Hua K. Huang, "Low cost Schemes For Fault tolerance in Matrix Operations With Processor Arrays," in *Proc. 12 th Int. Symp. on Fault-Tolerant computing*, Santa Monica, California, June 21-24, 1982.

[2] K. H. Huang and J. A. Abraham, "Algorithm-Based Fault Tolerance for Matrix Operations," *IEEE Transactions on Computers*, vol. C-33, pp. 518-528, June 1984.

[3] J. Y. Jou and J. A. Abraham, "Fault-Tolerant FFT Networks," *Proc. 15th Int. Symp. Fault-Tolerant Computing*, June 1985.

[4] P. Banerjee and J. A. Abraham, "Concurrent Fault Diagnosis in Multiple Processor Systems," in *16th Int. Symp. on Fault-Tolerant Computing*, Vienna, Austria, pp. 298-303, 1986.

[5] F. P. Preparata, G. Metze, and R. T. Chien, "On the Connection Assignment Problem of Diagnosable Systems," *IEEE Trans. on Electronic Comp.*, vol. EC-16, pp. 848-854, December 1967.

[6] J. D. Russel and C. R. Kime, "System Fault Diagnosis: Closure and Diagnosability with Repair," *IEEE Trans. on Comp.*, vol. C-24, pp. 1078-1088, 1973.

[7] J. D. Russel and C. R. Kime, "System Fault Diagnosis: Masking, Exposure, and Diagnosability Without Repair," *IEEE Trans. on Comp.*, vol. c-24, pp. 1155-1161, 1975.

[8] Jou Jing-Yang and J. A. Abraham, "Fault Tolerant Matrix Arithmetic and Signal Processing on Highly concurrent Computing Structures," *Proc. of the IEEE*, vol. 74 no.5, pp. 732-741, May 1986.

[9] P. Banerjee, J. T. Rahmeh, C. B. Stunkel, V. S. S. Nair, K. Roy, and J. A. Abraham, "An Evaluation of System-Level Fault Tolerance on the Intel Hypercube Multiprocessor," in *Proc. 18 th Int. Symp. on Fault-Tolerant Computing*, Tokyo, Japan , June 1988 (to appear).

[10] J. R. Samson Jr. and F. A. Horrigan, "The Advanced Onboard Signal Processor (AOSP) - A Valid Concept," *Proc. DARPA Strategic Space Symposium*, pp. 1-24, 1983.